

Task -1 :

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <fcntl.h> #include <unistd.h> int main() {

char command[1000];  char *append_pos;

printf("Append Shell\n");  printf("Type

commands using >> or type exit\n");

    while (1) {      printf("append$ ");      fflush(stdout);

if (fgets(command, sizeof(command), stdin) == NULL)

        break;

        command[strcspn(command, "\n")] = '\0';

if (strcmp(command, "exit") == 0)

        break;

        append_pos = strstr(command, ">>")

if (append_pos != NULL) {

*append_pos = '\0';      char *cmd =

command;      char *file =

append_pos + 2;
```

```
while (*file == ' ')    file++;

int fd = open(file, O_WRONLY | O_CREAT | O_APPEND, 0644);

if (fd < 0) {           perror("open");        continue;
}

int saved_stdout = dup(STDOUT_FILENO);        dup2(fd, STDOUT_FILENO);

close(fd);            system(cmd);            fflush(stdout);

dup2(saved_stdout, STDOUT_FILENO);            close(saved_stdout);

} else {

    system(command);

}

return 0;
}
```

Output:

```
ivikram@DESKTOP: ~/OSSP# nano append.c
ivikram@DESKTOP:~/OSSP# gcc append.c -o append
ivikram@DESKTOP:~/OSSP# ./append
Append Shell
Type commands using >> or type exit
append$ echo "First line" > test.txt
append$ echo "Second line" >> test.txt
append$ cat test.txt
First line
Second line
append$ echo "Third line" >> test.txt
append$ cat test.txt
First line
Second line
Third line
append$ exit
ivikram@DESKTOP:~/OSSP# |
```

Task - 2:

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h> int
main() { char
command[1000];
```

```

    printf("Stderr Redirection Shell\n");    printf("Use 2>
for error redirection\n");    printf("Type exit to quit\n");
while (1) {    printf("stderr$ ");    fflush(stdout);
if (fgets(command, sizeof(command), stdin) == NULL)
    break;

    command[strcspn(command, "\n")] = '\0';
if (strcmp(command, "exit") == 0)
    break;

    char *redirect = strstr(command, "2>");
if (redirect != NULL) {
        *redirect = '\0';        char *cmd =
command;        char *file = redirect + 2;
while (*file == ' ')        file++;        if
(*file == '\0') {        printf("Error: No
file specified\n");        continue;
    }

    int fd = open(file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
perror("open");
continue;
    }

    int old_stderr = dup(STDERR_FILENO);

```

```

        if (old_stderr < 0) {
perror("dup");
close(fd);
continue;
        }

        if (dup2(fd, STDERR_FILENO) < 0) {
perror("dup2");          close(fd);
close(old_stderr);        continue;
        }          close(fd);          int status =
system(cmd);          fflush(stderr);          if
(dup2(old_stderr, STDERR_FILENO) < 0) {
perror("restore stderr");
        }

        close(old_stderr);          if (status == -1) {
fprintf(stderr, "Command execution failed\n");
        }

    } else {

        system(command);

    }
}

return 0;
}

```

Output:

```
ivikram@DESKTOP: ~/OSSP  ×  +  ∨  
ivikram@DESKTOP:~/OSSP# nano stderr.c  
ivikram@DESKTOP:~/OSSP# gcc stderr.c -o stderr  
ivikram@DESKTOP:~/OSSP# ./stderr  
Stderr Redirection Shell  
Use 2> for error redirection  
Type exit to quit  
stderr$ stderr$ cat error.txt  
sh: 1: stderr$: not found  
stderr$ stderr$ ls /wrongpath 2> error.txt  
stderr$ stderr$ cat error.txt  
sh: 1: stderr$: not found  
stderr$ stderr$ cat /filedoesnotexist 2> error2.txt  
stderr$ stderr$ cat error2.txt  
sh: 1: stderr$: not found  
stderr$ exit  
ivikram@DESKTOP:~/OSSP# |
```